

Herald



Herald — commons_messaging Module Guide (Developer / Architecture)

Field	Value
Revision	1
Created	2026-05-31
Last modified	2026-05-31
Status	active
Status summary	Architecture/developer reference for commons_messaging — the channel-adapter + LLM-dispatch framework. Documents the TWO layered adapter contracts (the §11.0 commons.Channel outbound interface vs the richer channels.Channel inbound interface that embeds it and adds SendReplyGeneric + BotSelfIdentity + DownloadAttachment), the init()-registered name→constructor registry (channels.Register/New/Names), the channel-agnostic self-echo filter (channels.StampSender/IsSelfEcho), the per-channel content-addressed inbox (~/.herald/inbox/<channel>/<sha256>.<ext>), the three shipped adapters (null, tgram, slack), and the dispatch/claude_code/dispatcher (FormatEnvelopeWithPreText, the <<<HERALD-DISPATCH-v1>>> envelope, the Tier-2 intent-inference instruction, and <<<HERALD-REPLY>>> reply parsing). Operator-facing channel SETUP lives elsewhere (see §8) — this guide is the developer view of the code. ANTI-BLUFF: every section documents only what the source under commons_messaging/ actually does as of this revision.
Issues	(none specific to this guide) bump when Wave 8 lands the Max / Email adapters (each a third/fourth channels.Channel registered via init()), when the tgram native SendReply migrates to the generic signature (T5 scope, see §3.4 divergence note), and when the §11.14 null adapter is widened to satisfy the richer channels.Channel inbound interface (today it implements only commons.Channel).
Continuation	

Table of contents

- [§1. Overview](#)

- [§2. The two layered interfaces](#)
 - [§3. The shipped adapters](#)
 - [§4. The channel registry](#)
 - [§5. The self-echo filter and the content-addressed inbox](#)
 - [§6. The Claude Code dispatch flow](#)
 - [§7. How to add a new channel adapter](#)
 - [§8. References](#)
-

§1. Overview

`commons_messaging` (module path `github.com/vasic-digital/herald/commons_messaging`) is Herald's **channel-adapter + dispatch framework** — the layer that owns "how a notification reaches a messenger" (outbound) and "how a subscriber's message reaches Claude Code and a reply gets quoted back" (inbound). It is L1 in the layering convention: it imports `commons` (L0) and is imported by the flavor binaries (`pherald`, ...).

Two subtrees make up the package:

1. `channels/` — the adapter framework. The generic `channels.Channel` interface, the `init()`-registered constructor registry, the channel-agnostic self-echo filter, the per-channel content-addressed inbox, and the three concrete adapters: `channels/null/` (§11.14 sandbox), `channels/tgram/` (Telegram Bot API), `channels/slack/` (Slack Web API + Socket Mode).
2. `dispatch/claude_code/` — the LLM dispatcher. It formats the <<<HERALD-DISPATCH-v1>>> envelope, shells out to `claude --resume <UUID> --print "<envelope>"`, and parses the structured <<<HERALD-REPLY>>> action the model emits.

The central design decision is that there are **two adapter contracts**, not one — see §2. Outbound notification fan-out only needs `commons.Channel` (the spec §11.0 contract); the inbound runtime (`pherald listen`) needs a strictly richer set of methods, so `channels.Channel` **embeds** `commons.Channel` and adds the three inbound methods rather than widening the §11.0 contract (which would ripple into every flavor's outbound `serve` path).

§2. The two layered interfaces

§2.1 `commons.Channel` — the §11.0 outbound contract

Defined in `commons/types.go`. This is the interface every adapter must satisfy at minimum; it is all the outbound fan-out path depends on:

```
type Channel interface {
    Name()
        string //
        "tgram", "slack", ...
    Capabilities()
        Capabilities //
        declarative feature flags
    Send(ctx context.Context, msg OutboundMessage) (Receipt,
        error)
```

```

Subscribe(ctx context.Context, h InboundHandler)
    error // long-running, called by `serve`
HealthCheck(ctx context.Context) error
}

```

- `Name()` returns the canonical `ChannelID` string ("tgram", "slack", "null") — these match the `commons.ChannelID` constants and key the per-channel inbox.
- `Capabilities()` is a declarative feature-flag struct (`Text/Markdown/HTML/Attachments/AttachmentMaxMiB/Threads/InteractiveURL/InteractiveCall/DeliveryCeiling`). Routers consult it before dispatching; a mismatch is the adapter's choice to drop or downgrade.
- `Send` takes a fully-rendered `commons.OutboundMessage` and returns a `commons.Receipt` whose `Evidence` is the strongest `DeliveryEvidence` the channel can prove (accepted < routed < delivered < read).
- `Subscribe` is the long-running inbound loop driven by `serve`; it calls the supplied `commons.InboundHandler.Handle` for each subscriber message.

§2.2 `channels.Channel` — the richer inbound contract

Defined in `commons_messaging/channels/channel.go`. It **embeds** `commons.Channel` and adds the three methods the Wave 6 Telegram adapter implicitly defined and that pherald `listen` + `qaherald` depend on:

```

type Channel interface {
    commons.Channel

    SendReplyGeneric(ctx context.Context, recipient
        commons.Recipient, body string,
        replyToID string, attachments []commons.Attachment)
        (string, error)
    BotSelfIdentity(ctx context.Context) (SelfIdentity, error)
    DownloadAttachment(ctx context.Context, externalID string,
        mime string) (string, string, error)
}

```

- `SendReplyGeneric` posts a reply quoting the message `replyToID`, then fans each attachment out as its own reply at the same thread depth. `recipient.ChannelUserID` is the channel-native target (Telegram `chat_id`, Slack channel id). `replyToID == ""` is the no-reply sentinel (fresh message). Returns the channel-native message id of the text reply.
- `BotSelfIdentity` returns the channel-native bot identity. It **MUST** cross the wire on first call (`getMe / auth.test`); it **MAY** cache. An empty `Value` is an echo-loop hazard, so `Subscribe` refuses to boot without one.
- `DownloadAttachment` streams the channel-hosted file `externalID` into `~/herald/inbox/<channel>/<sha256>.<ext>` while hashing inline, returning (`finalPath, sha256Hex, error`). It is idempotent — content is proof of presence (see §5.2).

Why a separate package and not a wider `commons.Channel`? `commons.Channel` is the §11.0 contract every flavor's outbound `serve` path depends on; widening it would force every flavor to gain the inbound methods. The inbound runtime is a strictly richer need, so

`channels.Channel` composes the §11.0 contract instead. This is documented verbatim at the top of `channel.go`.

§2.3 The SelfIdentity discriminated union

The self-identity each channel reports is channel-native, so `channels.go` carries a tagged union:

```
type IdentityKind string
const (
    IdentityUsername IdentityKind = "username" // Telegram:
        bot.Me.Username (the @handle)
    IdentityUserID    IdentityKind = "user_id"  // Slack: the
        bot_user_id (e.g. "U01ABC")
    IdentityAddress   IdentityKind =
        "address" // Email: the From address Herald sends as
)

type SelfIdentity struct {
    Kind IdentityKind
    Value string // username (no @), user-id, or address — per
        Kind
}
```

The generalized self-filter (§5.1) compares the inbound sender against this without knowing the concrete channel.

§3. The shipped adapters

Adapter	Package	Interface satisfied	Transport
null	<code>channels/null/</code>	<code>commons.Channel</code> (outbound only)	in-process ring buffer (§11.14 sandbox)
tgram	<code>channels/tgram/</code>	full <code>channels.Channel</code> (inbound)	Telegram Bot API via <code>gopkg.in/telebot.v3</code>
slack	<code>channels/slack/</code>	full <code>channels.Channel</code> (inbound)	Slack Web API + Socket Mode via github.com/slack-go/slack

§3.1 null — the §11.14 sandbox

`channels/null/null.go` is the in-process equivalent of `/dev/null` with full instrumentation. Every `Send` records the `OutboundMessage` into a bounded ring buffer, increments per-outcome and per-tag counters, optionally fails per a `fail_rate`, optionally sleeps per `latency_ms`, and returns the configured `DeliveryEvidence` ceiling. Construct via `New("null://[?seed=...&fail_rate=0..1&latency_ms=...&ceiling=accepted|routed|delivered|read&tags=...]"`). It is used by routing unit tests, throughput load tests, quickstart/training, and chaos testing (set `fail_rate` to exercise retry/DLQ paths). It implements only `commons.Channel` today — its `Subscribe` is a no-op that blocks on `ctx.Done()`. It is NOT registered in the `channels` registry and MUST NOT be enabled in production.

§3.2 tgram — Telegram Bot API

`channels/tgram/tgram.go` plus the `send.go` / `subscribe.go` / `healthcheck.go` / `attachments.go` siblings. The adapter `lazy-constructs` its `*telebot.Bot` exactly once (`ensureBot + sync.Once`) — `telebot.NewBot` dispatches `getMe` during construction, so that single call IS the live roundtrip that populates `a.bot.Me`. Three constructors exist:

- `New(rawURL)` parses a `tgram://<bot_token>/<chat_id>[?tags=...]` URL. Note the §107 anti-bluff carve-out: real Telegram tokens are `<numeric_id>:<alphanumeric_hmac>` and the embedded colon trips Go's `url.Parse` host:port rule, so `New` short-circuits the canonical shape before `url.Parse`.
- `NewWithCreds(botToken, chatID)` is the recommended constructor when you already hold the credentials (e.g. pherald reading `HERALD_TGRAM_BOT_TOKEN` from `env`) — it bypasses `url.Parse` entirely.
- `NewWithStorage(rawURL, pool)` adds best-effort persistence of `outbound_delivery_evidence` rows after a successful send.

The three `channels.Channel` inbound methods (`BotSelfIdentity`, `SendReplyGeneric`, `DownloadAttachment`) are thin adapters over the Wave 6 substrate — see §3.4 for why `SendReplyGeneric` is named as it is.

§3.3 slack — Slack Web API + Socket Mode

`channels/slack/slack.go` plus `send.go` / `subscribe.go` / `selfidentity.go` / `attachments.go` / `healthcheck.go`. This is the SECOND concrete `channels.Channel`, proving the abstraction the Telegram adapter implicitly defined. Outbound rides the Slack Web API (`chat.postMessage`, `files.uploadV2`, `files.info`, `auth.test`); inbound rides **Socket Mode** (Slack's WebSocket equivalent of Telegram's `getUpdates` long-poll), which requires an app-level token (`xapp-...`) in addition to the bot token (`xoxb-...`). `BotSelfIdentity` resolves the `bot_user_id` (e.g. `U01ABC`) via `auth.test` and caches it. Constructors: `New(botToken, appToken, channelId)`, `NewWithBaseURL(..., baseURL)` (the `httptest` seam), and `NewFromURL("slack://<bot-token>@<workspace>/<channel-id>[?app_token=<xapp-...>]")`.

§3.4 The SendReplyGeneric naming divergence (load-bearing)

The Wave 7 plan text named the generic reply method `SendReply`, but `*tgram.Adapter` already holds a native `SendReply(ctx, chatID int64, body string, replyTo int, ...)` that is behaviour-critical and §107-mutation-gate-anchored — and a Go type may hold only ONE method named `SendReply`. The resolution (documented at the top of `channel.go`) is to name the **interface** method `SendReplyGeneric`. The `tgram` adapter adds `SendReplyGeneric` as a thin `string→int64/int` wrapper over its native `SendReply`; the `slack` adapter names its method `SendReplyGeneric` directly. The separate inbound `inbound.Replier` interface keeps the name `SendReply` independently (it is the inbound package's own name). Keep this in mind when adding adapters: implement `SendReplyGeneric`, not `SendReply`, to satisfy `channels.Channel`.

§4. The channel registry

`channels/registry.go` is a process-global name → Constructor map populated by each adapter's `init()`. It is what lets pherald listen resolve a channel by name at runtime without compile-time knowledge of the concrete adapter.

```
type Config struct {
    Channel string          // name being constructed
        ("tgram", "slack")
    Token string            // primary credential (Telegram
        token, Slack xoxb-)
    AppToken string         // secondary (Slack xapp- app-
        level token for Socket Mode)
    Target
        string              // default outbound dest (Telegram
        chat_id, Slack channel id, email)
    BaseURL string          // httptest seam; "" => live
        endpoint
    Extra map[string]string // channel-specific (e.g. email
        IMAP host/port)
}
type Constructor func(cfg Config) (Channel, error)

func Register(name string, ctor Constructor) // panics on
    duplicate
func New(name string, cfg Config) (Channel, error) // wraps
    ErrUnknownChannel when none
func Names() []string // every registered name, alphabetical
```

Function	Behaviour
<code>Register(name, ctor)</code>	Installs <code>ctor</code> under <code>name</code> . Panics on duplicate — a typo registering two adapters under one name would silently shadow one (a bluff class the framework forbids; mirrors <code>qaherald scenario.Register</code>). Called from each adapter's <code>init()</code> .
<code>New(name, cfg)</code>	Looks up the registered constructor, stamps <code>cfg.Channel = name</code> , and calls it. Returns a wrapped <code>ErrUnknownChannel</code> (including the list of registered names) when none matches — callers MUST surface it; a silent no-op channel is a §107 PASS-bluff.
<code>Names()</code>	Returns every registered name, alphabetically (used in the <code>ErrUnknownChannel</code> message and by operator tooling).

The `Config` is deliberately channel-agnostic — an adapter reads only the fields it understands. `tgram's init()` reads `cfg.Token` (bot token) + `cfg.Target` (`chat_id`); `slack's` reads `cfg.Token` (`xoxb-`) + `cfg.AppToken` (`xapp-`) + `cfg.Target` (`channel id`). Both honour `cfg.BaseURL` as the `httptest` seam.

§5. The self-echo filter and the content-addressed inbox

§5.1 Channel-agnostic self-echo filter

`channels/selffilter.go` generalizes the Wave 6 §32.9 anti-echo-loop guarantee so the inbound runtime drops its OWN bot's messages before re-dispatching a reply to them — without channel-specific knowledge. Two functions form a lock-step pair:

- `StampSender(raw, isBot, kind, value)` — adapters call this from their `Subscribe` handler after building the `InboundEvent`, passing `ev.Raw`. It records the sender's native identity (`sender_is_bot`, `sender_identity_kind`, `sender_identity`) into the raw map. A nil map is a no-op.
- `IsSelfEcho(ev, self)` — the runtime calls this with the bot's own `SelfIdentity`. It returns true only when the stamped sender is a bot AND its (`kind`, `value`) equals the bot's own identity.

The scope is deliberately narrow: a **different** bot in the same conversation is KEPT (multi-bot collaboration is real subscriber traffic). An empty `self.Value` never classifies as echo — a conservative KEEP that surfaces as duplicate traffic rather than silent message loss.

§5.2 Per-channel content-addressed inbox

`channels/inbox.go` owns the on-disk attachment store. Two helpers back every adapter's `DownloadAttachment`:

- `InboxDir(channel)` returns `~/.herald/inbox/<channel>/` (created 0700). Per-channel isolation lets the same sha256 from two channels coexist and makes the on-disk forensic trail self-describing by channel.
- `WriteContentAddressed(channel, mime, r)` streams `r` into `<inbox>/<sha256>.<ext>` while hashing inline via `io.MultiWriter` (never buffering the full payload), then atomically renames into place. Returns (`finalPath`, `sha256Hex`, `error`). **Idempotent**: if the content-addressed file already exists, the temp file is dropped and the existing path returned unchanged — same sha256 $\Rightarrow$ byte-equal content, so a duplicate poll costs zero quota. `MimeToExt(mime)` maps the small set of MIMEs subscribers actually send (photo/document/voice) to a canonical extension, falling back to `bin` so the file is always recoverable.

This is the channel-agnostic promotion of the Wave 6 tgram-private stream-hash-rename body, so every adapter shares one algorithm.

§6. The Claude Code dispatch flow

`dispatch/claude_code/` is the LLM dispatcher (spec §33). The `Dispatcher` (`claude_code.go`) resolves a per-project Claude Code session, formats the envelope, shells out, and parses the reply.

§6.1 Session resolution

`Dispatcher` is constructed via `New(binaryPath, workingDir, projectName)` or `NewWithStorage(..., pool)` (adds best-effort session persistence). `ResolveSession()` reads the anchor file `<workingDir>/~/.herald/claude-code/sessions/<projectName>.session`; a valid UUID there is resumed, otherwise `buildCmd` calls

bootstrapSession to spawn `claude --session-id <new-uuid>` non-interactively, persists the anchor, and proceeds. A bounded per-message deadline (`DefaultDispatchTimeout = 120s`, overridable via `HERALD_CLAUDE_DISPATCH_TIMEOUT` or `SetDispatchTimeout`) wraps the caller `ctx` via `context.WithTimeout` — the production caller (`pherald listen`) passes the unbounded long-poll `ctx`, so without this bound a hung `claude` would wedge the inbound goroutine forever (a §11.4/§107 resilience bluff). The whole process group is killed on `ctx-cancel`.

§6.2 The <<<HERALD-DISPATCH-v1>>> envelope

`buildCmd` runs `claude --resume <UUID> --model claude-opus-4-7 --print "<envelope>"` (Opus is operator-locked on the literal `argv`). The envelope is built by `FormatEnvelopeWithPreText(req, channelName)`, which prepends, before the structured block:

1. The **verbatim operator pre-text** — "We have received new message from our communication channel <name>. ..." (this opening sentence **MUST** appear as a strict prefix; `TestFormatEnvelopePreText` asserts via `strings.HasPrefix`).
2. The classification sentence, sender, inbound id, and attachment list (downloaded attachments carry their local `~/ .herald/inbox/<channel>/<sha256>.<ext>` path in the `Filename` field).
3. The **ACTION FORMAT GUIDANCE** block, which maps `classification.type` to the action the reply must use (`issue.open` for bug/task/implementation/investigation; reply for query/empty; `event.emit` for `event_trigger`).
4. The **Tier-2 intent-inference instruction** (`intentInferenceInstruction`, from `docs/design/INTENT_RECOGNITION.md` §4): it tells the LLM the user speaks PLAIN LANGUAGE — no command syntax, no `COMMAND:` prefix — to map natural language onto Herald's command set, and — crucially — to return `action=clarify` with a precise question instead of guessing when intent is not confidently determinable (§11.4.6: a wrong action is worse than a clarifying question). `TestFormatEnvelope_IntentInferenceInstruction` pins both this block and the literal `action=clarify` token.

The inner `FormatEnvelope(req)` emits the <<<HERALD-DISPATCH-v1>>> ... <<<END-HERALD-DISPATCH>>> structured block (Project / Inbound ID / Sender / Channel / Classification / Conversation / Attachments / User message) and the <<<HERALD-REPLY>>> JSON schema the model must answer with.

§6.3 Reply parsing

`Dispatch` calls `parseReply(stdout)`, which scans for the <<<HERALD-REPLY>>> marker and decodes the first JSON object after it into a `DispatchResponse` (`outcome/summary/details/affected_paths/reproduction_steps/estimated_effort/workable_item_proposed/follow_up_questions`). §107 anti-bluff: a PASS requires (a) `claude` exits 0 AND (b) `stdout` carries a well-formed JSON reply on a <<<HERALD-REPLY>>> line — a missing marker or malformed JSON is an explicit FAIL; no defaults are synthesised.

§6.4 Where action routing happens

The dispatcher itself stops at parsing. The **action routing** — `reply` → `Replier.SendReply`, `issue.open` → `IssueOpener.OpenIssue`, `event.emit` →

EventEmitter.Emit, unknown → explicit error — lives in pherald/internal/inbound/dispatcher.go (the commons.InboundHandler the adapter's Subscribe loop drives). The Tier-1 deterministic fast-path (Help/Status/Continue/Done/Reopen and the natural-language CommandRecognizer) also lives there and never reaches the LLM. See the §11.4.105 intent-recognition contract for the three-tier resolution.

§7. How to add a new channel adapter

To add (say) a Max or Email adapter so pherald listen and qaherald pick it up with zero changes to their core loops:

1. **Create the package** `commons_messaging/channels/<name>/` and a `*Adapter` type.
2. **Satisfy** `commons.Channel` — implement `Name()` (return the `commons.ChannelID` string), `Capabilities()`, `Send`, `Subscribe`, `HealthCheck`. `Name()` MUST match the registry name so `channels.InboxDir(name)` keys correctly.
3. **Satisfy the inbound** `channels.Channel` — add `SendReplyGeneric` (NOT `SendReply` — see §3.4), `BotSelfIdentity` (return the right `SelfIdentity.Kind` for the channel; never an empty `Value`), and `DownloadAttachment` (route through `channels.WriteContentAddressed(name, mime, r)` so you inherit the content-addressed, idempotent inbox).
4. **Stamp the sender** in your `Subscribe` handler via `channels.StampSender(ev.Raw, isBot, kind, value)` so the channel-agnostic `IsSelfEcho` filter works without channel-specific code.
5. **Register** in an `init()`:
`channels.Register(string(common.ChannelName), func(cfg channels.Config) (channels.Channel, error) { ... })`, reading only the `Config` fields your channel needs. Add the matching `httptest-seam` branch on `cfg.BaseURL`.
6. **Add a compile-time interface assertion** in a test: `var _ channels.Channel = (*Adapter)(nil)`.
7. **Hermetic tests** — drive every wire method through an `httptest` server and count the round-trips (a method that compiles but never hits the wire is a §107 bluff). Land a `docs/qa/<run-id>/transcript` per §107.x.

The flavor binaries never change: they call `channels.New(name, cfg)` and operate on the returned `channels.Channel`.

§8. References

- **Source.** `commons_messaging/channels/channel.go` (the two-interface design + `SelfIdentity`), `channels/registry.go` (registry), `channels/selffilter.go` (self-echo), `channels/inbox.go` (content-addressed inbox), `channels/null/null.go`, `channels/tgram/tgram.go`, `channels/slack/slack.go`, `dispatch/claude_code/claude_code.go` + `dispatch.go`.
- **L0 contract.** `commons/types.go` — `Channel`, `Capabilities`, `OutboundMessage`, `Receipt`, `InboundEvent`, `InboundHandler`, `Recipient`, `Attachment`, the `ChannelID` constants.
- **Inbound routing.** `pherald/internal/inbound/dispatcher.go` (action routing, the `Replier/IssueOpener/EventEmitter` sinks) and `command_recognizer.go` (Tier-1 fast-path).

- **Operator-facing channel SETUP (do not duplicate here):**
 - docs/guides/MESSENGER_CHANNELS.md — the cross-channel setup matrix.
 - docs/guides/TELEGRAM.md + docs/guides/MTPROTO.md — Telegram bot + MTPROTO user-client setup.
 - docs/guides/messengers/SLACK.md (and the per-channel siblings MAX.md, EMAIL.md, ...) — per-messenger setup.
 - docs/guides/OPERATOR_CREDENTIALS.md — env-var + credential resolution.
- **Spec.** docs/specs/mvp/specification.V4.md §11 (the commons.Channel §11.0 contract + §11.1 Telegram + §11.14 null), §32 (inbound pipeline + §32.9 anti-echo), §33 (Claude Code dispatch + the <<<HERALD-DISPATCH-v1>>> / <<<HERALD-REPLY>>> envelope).
- **Intent recognition.** docs/design/INTENT_RECOGNITION.md (the Tier-2 intent-inference instruction baked into the envelope, the Tier-3 action="clarify" fallback) — Herald §110 / Helix §11.4.105.
- **Dependencies.** gopkg.in/telebot.v3 (tgram), github.com/slack-go/slack (slack).

Sources verified

This guide documents internal Herald source only; no external service/library online documentation was relied on. All behavioural claims are grounded in the cited source files under commons_messaging/ and commons/types.go as of 2026-05-31.

Verified 2026-05-31: internal doc — no external online sources. Behavioural claims derive from commons_messaging/channels/{channel,registry,selffilter,inbox}.go, channels/{null,tgram,slack}/*.go, dispatch/claude_code/{claude_code,dispatch}.go, and commons/types.go (all read 2026-05-31). The only third-party dependencies are gopkg.in/telebot.v3 and github.com/slack-go/slack, both pinned in the respective adapter go.mod files — the API surface used is the vendored, version-pinned one (no online-doc cross-reference required). Re-verify on a telebot/slack-go major-version bump, when the Max/Email adapters land, or on any channels.Channel / commons.Channel interface change.